geometry is still blocking both sides of your view, the clip didn't go deep enough. dk64.fandom.com

10. From inside the wall, move toward the boss-door/loading-zone side of the room.
11. Walk into the boss loading zone from behind/through the geometry.
12. You should load directly into the Crystal Caves boss fight without paying the full colored-banana requirement. Ledge clips are specifically documented as a way to enter boss fights from Troff 'n' Scoff without collecting the required bananas. dk64.fandom.com +1

The **hard part is Step 7**. The timing is basically "C-Up as the Kong transitions from standing to falling," not after you are already clearly dropping.

Also, since you're on a current build, there's a newer **frame-clip method** that some players find easier than the old ledge clip: get onto the Troff 'n' Scoff portal frame, move toward the center, clip through the wall, then jump before entering the falling state. It works for **Diddy, Lanky, Tiny, and DK**, but not Chunky. YouTube

If you tell me **which Kong the Crystal Caves boss door is asking for in your seed**, I can tell you whether the ledge clip or the newer frame clip is probably easier for that Kong.

You said:

Okay the archipelago is a successful experiment. But what if I wanted to create a package that I could give to my friends so we can set up any amount of archipelagos easier

ChatGPT said:

Yes. You can make a **portable "Archipelago Party Kit"** that turns the process you just went through into something much closer to:

unzip → run setup → choose games/players → generate → play

The main limitation is that you should **not bundle ROMs/ISOs**. Each player should provide their own legally obtained game image. Archipelago itself also expects ROM-based games to locate a base ROM during generation when needed. Archipelago

The structure I'd recommend is:

```
Archipelago-Party-Kit/
│
├── Setup.bat
├── Generate-Game.bat
├── Start-Server.bat
├── README.txt
│
├── apworlds/
│   ├── dk64.apworld
```

```
│   ├── Twilight Princess.apworld
│   └── ...
│
├── templates/
│   ├── DK64/
│   │   ├── moderate.yaml
│   │   ├── easy.yaml
│   │   └── chaos.yaml
│   │
│   ├── TwilightPrincess/
│   │   ├── moderate.yaml
│   │   └── easy.yaml
│   │
│   └── ...
│
├── players/
│
├── tools/
│   ├── tp-runtime/
│   └── trackers/
│
└── config/
    └── party-settings.json
```

The important design choice is that **the host kit and player kit should be slightly different**.

Host package

Your copy would contain the generator workflow and all the APWorlds your group commonly plays.

Archipelago officially supports installing custom worlds by feeding `.apworld` files into the launcher or placing them into `custom_worlds`; those custom worlds can then be generated locally like normal games. Archipelago

Your setup script could therefore:

1. Detect where Archipelago is installed.
2. Check that the required AP version is installed.
3. Copy/install your known-good `.apworld` versions.
4. Back up existing `Players` files.
5. Ask:

   ```
   How many players?
   ```

6. For each person:

   ```
   Player name:
   Game:
   ```

```
Preset:
```

7. Copy the appropriate YAML template.
8. Replace:

```
name: TEMPLATE
```

   with the entered player name.
9. Put all generated YAMLs into Archipelago's `Players` directory.
10. Run the Archipelago generator.
11. Locate the resulting:

```
AP_XXXXX.zip
```

12. Optionally launch:

```
ArchipelagoServer.exe AP_XXXXX.archipelago
```

That would eliminate most of the repetitive manual setup.

Player package

Then each friend could receive a smaller package containing only the pieces needed for their game.

For example, Josh's TP package could look like:

```
Josh-TP/
│
├── SETUP-TP.bat
├── START-TP.bat
│
├── apworld/
│   └── Twilight Princess.apworld
│
├── dolphin-files/
│   ├── GZ2E01_REL_Loader_v2.gci
│   ├── RandomizerAP.us.gci
│   └── Tpr-E-APTest_APT-aptest.gci
│
└── README.txt
```

The current Twilight Princess setup requires those three GCI runtime files in Dolphin's GCI-folder setup, plus the TP APWorld. GitHub

Your script could install those automatically after asking Josh where his Dolphin user folder is.

Then `START-TP.bat` could launch:

```
Dolphin
```

```
      +
Twilight Princess Client
      +
PopTracker
```

all at once.

Your DK64 package

Likewise:

```
Tamas-DK64/
│
├── SETUP-DK64.bat
├── START-DK64.bat
│
├── apworld/
│   └── dk64.apworld
│
├── tracker/
│   └── dk64-tracker-pack.zip
│
└── README.txt
```

The one thing the script **should not contain** is your DK64 ROM.

Instead, on first run it could say:

```
Please select your legally obtained US DK64 ROM.
```

Then remember its path locally.

That fits Archipelago's own model: ROM games may require the generator to locate the base ROM the first time. Archipelago

---

The really nice version

Rather than having your friends edit YAML by hand, you could build a small launcher.

Something like:

```
╔═══════════════════════════════════╗
║       ARCHIPELAGO PARTY KIT       ║
╠═══════════════════════════════════╣
║ Players: 4                        ║
║                                   ║
```

```
╠ Tamas        DK64                  ╣
╠ Josh         Twilight Princess     ╣
╠ Sam          Ocarina of Time       ╣
╠ Mike         Super Metroid         ╣
╠                                     ╣
╠ Preset: Moderate                   ╣
╠                                     ╣
╠ [ Generate Game ]                  ╣
╚═════════════════════════════════════╝
```

Click **Generate Game**, and it builds all four YAMLs and invokes Archipelago.

You could write that in **Python** very easily. If you wanted it to feel like a normal Windows app, Python + Tkinter is enough.

You wouldn't even need everyone to install Python if you package it into an `.exe` using something like PyInstaller.

Presets would be the biggest quality-of-life improvement

Instead of teaching everybody every randomizer option, maintain presets like:

```
Easy
Moderate
Chaos
Custom
```

For TP, for example, your group's `moderate.yaml` could already have:

```
logic_rules:
  glitchless: 50
  glitched: 0

small_key_settings:
  anywhere: 50

big_key_settings:
  anywhere: 50

map_and_compass_settings:
  startwith: 50

start_inventory:
  Shadow Crystal: 1
```

```
exclude_locations:
  - Cave of Ordeals Floor 17 Poe
  - Cave of Ordeals Floor 33 Poe
  - Cave of Ordeals Floor 44 Poe
  - Cave of Ordeals Great Fairy Reward
```

Then Josh never has to edit that again.

He just selects:

```
Twilight Princess → Moderate
```

and your tool copies that YAML and inserts:

```
name: Josh
```

Version locking is important

This is probably the **most important engineering decision**.

Don't make your kit blindly download "latest."

Instead, make a manifest:

```
{
  "archipelago": "0.6.8",
  "worlds": {
    "dk64": "1.5.8",
    "twilight_princess": "known-good-version"
  }
}
```

Custom APWorlds execute code inside Archipelago, and Archipelago itself warns users to only install APWorlds from trusted sources. Archipelago

TP's own documentation also warns that mismatched seed/client APWorld versions can make seeds unplayable. GitHub

So your kit should basically say:

> **These exact versions are the versions our group tested.**

Then you intentionally update them later.

That will save you a lot of headaches.

You could also automate your self-hosting

Your current process is:

```
Generate
↓
find AP zip
↓
Host
↓
port 38281
↓
friends connect
```

Your launcher could reduce that to:

```
Generate & Host
```

Internally:

```
generate YAMLs
      ↓
run ArchipelagoGenerate
      ↓
find newest output
      ↓
run ArchipelagoServer
      ↓
show:
Server running on port 38281
```

Then show everyone a connection string:

```
YOUR-IP:38281
```

I would **not automate router configuration**. UPnP is possible but introduces enough networking/security weirdness that I'd keep your existing TCP `38281` forwarding as a one-time manual setup.

And you can make it support arbitrary player counts

Archipelago itself is already designed for arbitrary multiplayer combinations. Local generation can exceed the website generator's 30-player limit. Archipelago

So your program doesn't have to care whether you have:

```
2 players
```

or:

```
12 players
```

It simply generates one YAML per player:

```
Players/
├── Tamas_DK64.yaml
├── Josh_TP.yaml
├── Steve_SM.yaml
├── Alex_OOT.yaml
├── Rachel_ALttP.yaml
└── ...
```

and invokes the generator.

That's actually a very clean project.

What I would build

I'd make it a small Windows application called something like:

AP Party Launcher

with four screens:

```
1. Setup
   Archipelago path
   ROM paths
   emulator paths

2. Players
   Add/remove players
   Pick game
   Pick preset

3. Generate
   Validate YAML
   Generate multiworld
   Create patches

4. Play
   Start Server
   Start Clients
   Start Trackers
```

And store each person's machine-specific settings locally:

```
config/
   Tamas.json
   Josh.json
```

while keeping the shared preset files in Git.

That gives you something you could put on **GitHub**, zip up for your friends, and improve every time you add a new game.

The only things you'd intentionally keep out of the repository would be **ROMs/ISOs, personal saves, generated seeds, and probably any redistributable files whose licenses don't permit bundling**.

This is very achievable; most of the hard work you just did manually is deterministic enough to automate.

You said:

I would like to build a launcher for this, one that sets up the randomization for each player, minus patching the rom files as I think that might not be achievable. Sets up the yaml files and the archipelago files fairly easy.

Worked for 11s

ChatGPT said:

I'd build this as a **Windows-first Archipelago Party Launcher** that handles everything up through generation and hosting:

```
Choose players
    ↓
Choose each player's game
    ↓
Configure randomizer settings
    ↓
Install/verify required APWorlds
    ↓
Generate each player's YAML
    ↓
Generate the Archipelago multiworld
    ↓
Start the Archipelago server
    ↓
Show each player what files/instructions they need
```

ROM/ISO patching stays outside the launcher.

Recommended technology

Since this is Windows-focused, I'd use **C# + .NET 8 + WPF**.

That gives you:

- a normal standalone `.exe`
- no Python installation for your friends
- easy process launching
- easy Windows path handling
- straightforward JSON configuration
- mature YAML libraries such as `YamlDotNet`
- easy packaging into one distributable application

If you eventually want Linux/macOS support, Avalonia would be an alternative, but WPF is simpler for your curre use case.

Core design

The most important architectural decision is: **do not hard-code DK64 and Twilight Princess into the application logic.**

Make every supported game a definition.

For example:

```
ArchipelagoPartyLauncher/
│
├── Launcher.exe
│
├── config/
│   └── settings.json
│
├── games/
│   ├── dk64/
│   │   ├── game.json
│   │   ├── template.yaml
│   │   └── dk64.apworld
│   │
│   └── twilight-princess/
│       ├── game.json
│       ├── template.yaml
│       └── Twilight Princess.apworld
│
├── presets/
```

```
│   ├── dk64/
│   │   ├── standard.yaml
│   │   └── chaos.yaml
│   │
│   └── twilight-princess/
│       ├── standard.yaml
│       └── streamlined.yaml
│
└── generated/
```

Then adding another game later is mostly adding another folder.

A game definition

For example, Twilight Princess could have:

```
{
  "id": "twilight_princess",
  "displayName": "The Legend of Zelda: Twilight Princess",
  "archipelagoGameName": "Twilight Princess",
  "apworld": "Twilight Princess.apworld",
  "minimumArchipelagoVersion": "0.6.1",
  "template": "template.yaml",
  "outputExtension": ".aptp",
  "requiresPatch": false
}
```

DK64 might be:

```
{
  "id": "dk64",
  "displayName": "Donkey Kong 64",
  "archipelagoGameName": "Donkey Kong 64",
  "apworld": "dk64.apworld",
  "minimumArchipelagoVersion": "0.6.5",
  "template": "template.yaml",
  "outputExtension": ".chunky",
  "requiresPatch": true
}
```

The launcher doesn't need to understand what `.chunky` does.

It just says:

> Your DK64 seed was generated. Use this `.chunky` file with DK64 Randomizer.

That separation will save you a lot of trouble.

Main UI

I'd make the first screen look roughly like:

```
┌──────────────────────────────────────────────┐
│ Archipelago Party Launcher                   │
├──────────────────────────────────────────────┤
│                                              │
│ Players                                      │
│                                              │
│ Tamas      Donkey Kong 64       [Configure] [X]  │
│ Josh       Twilight Princess    [Configure] [X]  │
│                                              │
│              [+ Add Player]                  │
│                                              │
│                                              │
│ Archipelago: 0.6.8 ✓                         │
│ Required APWorlds: Installed ✓               │
│                                              │
│               [ Generate Multiworld ]          │
└──────────────────────────────────────────────┘
```

`Add Player` could support any number of players.

Player configuration

Clicking **Configure** opens game-specific settings.

For Josh:

```
Josh — Twilight Princess

Preset:
[ Streamlined ▼ ]

General
```

```
[x] Glitchless
[x] Shuffle overworld
[x] Shuffle dungeons
[x] Shuffle NPC items
[x] Shuffle shops

Dungeon Items
Small Keys:  [ Anywhere ▼ ]
Big Keys:    [ Anywhere ▼ ]
Maps:        [ Start With ▼ ]

Starting Items
[x] Shadow Crystal

Excluded Locations
[x] Cave of Ordeals

Goal
Castle Requirement:
[ Mirror Shards ▼ ]

                    [ Save ]
```

And the program generates the YAML.

Nobody ever edits YAML manually again.

Don't expose every YAML field initially

This is important.

Your first version shouldn't attempt to automatically render all 100+ settings available in arbitrary APWorlds.

Instead, each game definition should specify the settings you actually want to expose.

Something like:

```
{
  "settings": [
    {
      "yamlPath": "Twilight Princess.logic_rules",
      "label": "Logic",
      "type": "choice",
      "choices": [
```

```
        "glitchless",
        "glitched"
      ]
    },
    {
      "yamlPath": "Twilight Princess.early_shadow_crystal",
      "label": "Early Shadow Crystal",
      "type": "boolean"
    }
  ]
}
```

That lets you build a generic settings renderer later.

Even better: use presets first

For version 1, I would actually avoid dynamically editing dozens of properties.

Have presets:

```
Twilight Princess
    Vanilla-ish
    Standard
    Streamlined
    Chaos


DK64
    Standard
    Randomized
    Chaos
```

Each preset is already a valid YAML.

When Josh selects:

```
Twilight Princess
Streamlined
```

your program:

1. copies `streamlined.yaml`
2. changes:

   ```
   name: TEMPLATE
   ```

to:

```
name: Josh
```

3. applies any small overrides selected in the UI

That's extremely reliable.

YAML editing

Use **YamlDotNet** instead of doing string replacement for most settings.

Conceptually:

```
var yaml = LoadYaml(template);

yaml["name"] = player.Name;

yaml["Twilight Princess"]["start_inventory"]["Shadow Crystal"] = 1;

SaveYaml(yaml);
```

This avoids indentation mistakes like the ones we were manually worrying about earlier.

That's one of the biggest benefits of this launcher.

Generate temporary player YAMLs

Don't directly clutter the user's real Archipelago `Players` directory permanently.

Your program should maintain something like:

```
generated/
└── session-2026-10-24-001/
    ├── players/
    │   ├── Tamas_DK64.yaml
    │   └── Josh_TP.yaml
    │
    └── output/
```

Before generating, it can copy those into Archipelago's expected location or invoke the generator using that folc if the installed version supports specifying a player directory.

After generation, you've retained an exact record of the configuration used.

That is enormously useful when someone later asks:

> What settings did we use last time?

Save a session manifest

Every generated Archipelago should also get your own manifest:

```
{
  "name": "Saturday Night AP",
  "created": "2026-10-24T20:00:00",
  "archipelagoVersion": "0.6.8",

  "players": [
    {
      "name": "Tamas",
      "game": "dk64",
      "preset": "standard"
    },
    {
      "name": "Josh",
      "game": "twilight_princess",
      "preset": "streamlined"
    }
  ]
}
```

Then the launcher can have:

```
Recent Games

Saturday Night AP
Tamas — DK64
Josh — Twilight Princess
[Resume Server]

Last Week's AP
Tamas — DK64
Josh — TP
Mike — OOT
[Resume Server]
```

This would be a **huge improvement** over hunting for `AP_934824923.zip` .

APWorld management

I'd make this a first-class feature.

Have a screen:

```
Archipelago Environment

Archipelago
0.6.8                ✓

DK64 APWorld
1.5.8                ✓

Twilight Princess
0.x.x                ✓

[ Repair Environment ]
```

Your package contains the exact APWorld versions you've tested.

When the user clicks **Repair Environment**, your program installs/copies the known-good versions.

Don't automatically download newest versions.

Instead, version your Party Launcher releases:

```
Party Launcher 1.0

Archipelago 0.6.8
DK64 APWorld 1.5.8
TP APWorld x.x.x
```

That gives your friends identical environments.

Generation workflow

When the user clicks:

Generate Multiworld

the program should perform validation first.

```
Checking environment...

✓ Archipelago found
✓ Version compatible
✓ DK64 APWorld installed
✓ Twilight Princess APWorld installed
✓ Tamas DK64 YAML valid
✓ Josh TP YAML valid
✓ Player names unique

Generating...
```

Then invoke Archipelago's generator.

After completion:

```
Multiworld generated successfully.

Seed:
AP_483019294

Files:

Tamas
Donkey Kong 64
Tamas.chunky
[Open Folder]

Josh
Twilight Princess
Josh.aptp
[Open Folder]

[ Start Server ]
```

For TP you could additionally display:

> No patch file action is required for normal TP AP play.

For DK64:

> Open this `.chunky` using the DK64 DEV Randomizer.

Server management

This is also worth integrating.

Instead of manually opening Host:

```
[ Start Server ]
```

Your launcher starts the Archipelago server process with the generated game.

Then show:

```
Server Running

Local:
localhost:38281

LAN:
192.168.1.42:38281

External:
[Configure manually]

Players:

✓ Tamas — DK64
✓ Josh — Twilight Princess

[ Save Server ]
[ Stop Server ]
```

I would **not** have the launcher automatically configure port forwarding.

Keep that manual.

You could store:

```
{
  "serverPort": 38281
}
```

and remind the host:

> TCP 38281 must be forwarded for remote players.

Player handoff packages

This is where your idea gets especially useful.

After generation, your launcher could create:

```
PlayerPackages/
├── Tamas/
│   ├── README.txt
│   └── Tamas.chunky
│
└── Josh/
    └── README.txt
```

Josh doesn't actually need his `.aptp` for normal TP use, so his instructions could simply say:

```
Archipelago Party
-----------------

Player: Josh
Game: Twilight Princess

Server:
123.123.123.123:38281

Startup:

1. Start Dolphin.
2. Start Twilight Princess.
3. Run REL Loader v2 once.
4. Load save "Josh".
5. Start Twilight Princess Client.
6. Connect to the server above.
```

You could ZIP each directory:

```
Tamas_AP.zip
Josh_AP.zip
```

Then hand the appropriate package to each person.

What the launcher should NOT do

I'd deliberately keep these outside version 1:

- ROM distribution
- ISO distribution
- automatic ROM downloading
- ROM patching
- emulator installation
- automatic router changes
- automatic UPnP
- arbitrary APWorld downloading
- trying to understand every APWorld ever made

That keeps the project manageable and legally/technically clean.

MVP scope

I would build version **0.1** with exactly five features:

1. **Detect Archipelago**
2. **Manage bundled APWorlds**
3. **Add players + choose game/preset**
4. **Generate YAMLs**
5. **Run Archipelago generation**

Then version **0.2**:

6. Start/stop server
7. Save/resume sessions
8. Create player handoff packages

Then version **0.3**:

9. Editable game settings UI
10. PopTracker integration
11. Emulator/client launch shortcuts

That progression keeps the first version small enough that you can actually finish it.

The architecture I'd use

```
ArchipelagoPartyLauncher
│
├── Models
│   ├── Player.cs
```

```
│   ├── GameDefinition.cs
│   ├── GamePreset.cs
│   └── Session.cs
│
├── Services
│   ├── ArchipelagoLocator.cs
│   ├── ApWorldManager.cs
│   ├── YamlGenerator.cs
│   ├── MultiworldGenerator.cs
│   ├── ServerManager.cs
│   └── SessionManager.cs
│
├── Views
│   ├── MainWindow.xaml
│   ├── PlayerEditor.xaml
│   ├── GameSettings.xaml
│   └── GenerationResult.xaml
│
├── games
│   └── ...
│
└── presets
    └── ...
```

That is simple enough for one developer but won't become a mess once you support ten games.

Where I would start

The first concrete milestone should be:

> **A WPF app where you click Add Player → enter `Josh` → select Twilight Princess → click Generate YAML → and it produces the exact valid TP YAML you used in this seed.**

Then add DK64.

Once those two work, invoking Archipelago's generator is relatively small compared with everything else.

If you want, **we can start building the actual C#/.NET project next**, beginning with the solution structure, player, game data model, and a working WPF screen for adding arbitrary players.

You said:

How can I save this chat as a file?

ChatGPT said:

The most reliable built-in way is to **export your ChatGPT data**, which includes your conversation history. On web: **Profile → Settings → Data controls → Export data → Export**. ChatGPT then sends you a download link; the ZIP includes your chat history, including conversation JSON files. OpenAI Help Ce... +1

If you only want **this one chat as a readable file**, the quicker method is usually your browser's **Print → Save as PDF**. That preserves the conversation in one file without exporting your entire account.

For this launcher project, I'd probably save this conversation as a **PDF** for easy reading, and optionally do the full data export if you want a machine-readable copy too.